

Frontend Interview Handbook — Part 2: TypeScript & React Fundamentals

SECTION A: TYPESCRIPT

Q1: Why use TypeScript over JavaScript? What problem does it actually solve?

Answer: TypeScript adds static type checking at compile time, catching a class of bugs (wrong argument types, typos in property names, null/undefined access) before code ever runs, instead of discovering them at runtime or in production. It also dramatically improves editor tooling — autocomplete, inline docs, safe refactors (rename a property and every usage errors until fixed) — and serves as living documentation of function/data shapes for teammates.

Why asked: Filter question — weak answers say "it catches errors," strong answers connect it to team velocity, refactor safety, and self-documenting APIs.

Common mistake: Treating TS as "JS with some `: string` annotations sprinkled in" rather than actually modeling data shapes with interfaces/types and letting the compiler do work for you.

Q2: `interface` vs `type` — what's the actual difference, and when do you choose one over the other?

Answer: Both can describe object shapes and are largely interchangeable for that. Key differences: `interface` supports **declaration merging** (multiple `interface Foo {}` declarations combine automatically) and is generally preferred for object/class shapes that might be extended. `type` can represent unions, intersections, tuples, and mapped types that `interface` cannot (`type Status = 'idle' | 'loading' | 'error'`). Common convention: `interface` for public object/component prop shapes, `type` for unions, function signatures, and utility type compositions.

```
ts
interface User { id: string; name: string; }
type Status = 'idle' | 'loading' | 'error';
```

Q3: What are generics, and why do they matter? Write a generic `useFetch<T>` type signature.

Answer: Generics let you write reusable code that works across multiple types while still preserving type safety — instead of typing something as `any` (losing all safety) or writing near-duplicate functions per type.

ts

```
function useFetch<T>(url: string): { data: T | null; loading: boolean; error: Error } {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<Error | null>(null);
  // ... fetch logic
  return { data, loading, error };
}

// usage
const { data } = useFetch<User>('/api/users/1');
// data is typed as User | null, not `any`
```

Why asked: Since you list TypeScript as a skill, expect this to be probed directly — "write this on the spot" is common.

Q4: What's the difference between `any`, `unknown`, and `never`?

Answer: `any` opts out of type checking entirely — assignable to/from anything, defeats the purpose of TS, should be avoided. `unknown` is the type-safe counterpart — you can assign anything to it, but you must narrow its type (via a type guard or assertion) before you can use it, forcing you to actually check before acting. `never` represents a value that can never occur — used for functions that always throw, or exhaustiveness checks in switch statements.

ts

```
function processInput(input: unknown) {
  if (typeof input === 'string') {
    console.log(input.toUpperCase()); // safe, narrowed
  }
}
```

Common mistake: Using `any` as an escape hatch under deadline pressure and never coming back to fix it — interviewers sometimes ask directly, "when is `any` acceptable?" (Answer: rarely — third-party untyped libraries as a last resort, ideally wrapped and narrowed immediately at the boundary.)

Q5: Explain utility types: `Partial`, `Pick`, `Omit`, `Record`.

Answer:

- `Partial<T>` — makes all properties optional (useful for update/patch functions).

- `Pick<T, K>` — constructs a type with only the specified keys.
- `Omit<T, K>` — constructs a type excluding the specified keys.
- `Record<K, V>` — constructs an object type with keys of type `K` and values of type `V`.

ts

```
interface User { id: string; name: string; email: string; }

type UserUpdate = Partial<User>;           // all optional
type UserPreview = Pick<User, 'id'|'name'>; // only id, name
type PublicUser = Omit<User, 'email'>;     // everything but email
type UsersById = Record<string, User>;     // { [id: string]: User }
```

Q6: How do you type a React component's props, including children and optional/default values?

tsx

```
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: 'primary' | 'secondary'; // optional, union-constrained
  children?: React.ReactNode;
}

function Button({ label, onClick, variant = 'primary', children }: ButtonProps) {
  return <button className={variant} onClick={onClick}>{label}{children}</button>
}
```

Follow-up: "Why `React.ReactNode` for `children` instead of `JSX.Element`?" — `ReactNode` is broader (accepts strings, numbers, arrays, fragments, null, not just a single JSX element), which is almost always what you actually want for a `children` prop.

SECTION B: REACT FUNDAMENTALS

Q7: What is the virtual DOM, and why does it help performance?

Answer: The virtual DOM is a lightweight in-memory representation of the actual DOM. When state changes, React builds a new virtual DOM tree, **diffs** it against the previous one (reconciliation), and computes the minimal set of real DOM mutations needed — then applies only those. Direct DOM manipulation is expensive (layout/reflow/repaint), so batching and minimizing real DOM writes is the core performance win. Note: the virtual

DOM isn't "faster than the DOM" in some absolute sense — it's a strategy for minimizing expensive DOM writes and enabling a declarative programming model.

Q8: Explain the React component lifecycle in functional components using hooks (compared to class lifecycle methods).

Answer: Function components don't have lifecycle *methods*, but `useEffect` covers the same needs:

- Mount-only logic (like old `componentDidMount`): `useEffect(() => {...}, [])`
- Update logic tied to specific values (like a subset of `componentDidUpdate`): `useEffect(() => {...}, [dep1, dep2])`
- Cleanup (like `componentWillUnmount`): return a function from the effect.

```
jsx

useEffect(() => {
  const sub = subscribeToSomething();
  return () => sub.unsubscribe(); // cleanup
}, []);
```

Common mistake: Thinking of `useEffect` as "the mount/unmount hook" rather than "synchronize with an external system whenever these values change" — the mental model shift matters and interviewers listen for it.

Q9: Controlled vs uncontrolled components — what's the difference, and which do you prefer?

Answer: A **controlled** component's value is driven entirely by React state — the input's `value` comes from state and every change goes through `onChange` to update it, so React is the single source of truth. An **uncontrolled** component manages its own state internally in the DOM, and you read its value via a `ref` when needed (e.g., on submit). Controlled is generally preferred for forms needing validation, conditional logic, or synced UI; uncontrolled can be simpler/more performant for very large or simple forms where you only need the value at submit time.

```
jsx
```

```
// controlled
<input value={name} onChange={e => setName(e.target.value)} />

// uncontrolled
const inputRef = useRef();
<input ref={inputRef} defaultValue="" />
// later: inputRef.current.value
```

Q10: What are React keys, and what specifically goes wrong with `index` as a key?

Answer: Keys let React match array items between renders to decide what to update/move/remove, rather than re-rendering the whole list. Using array index works only if the list is static (never reordered, filtered, or has items inserted/removed). If items can move, index-based keys cause React to associate the wrong internal state (like an input's current text, or a component's local `useState`) with the wrong visual row — a very specific, hard-to-spot class of bug.

```
jsx

// risky if list reorders
{items.map((item, i) => <Row key={i} {...item} />)}

// correct - stable identity
{items.map(item => <Row key={item.id} {...item} />)}
```

Q11: Explain `useMemo` and `useCallback` with a concrete case where skipping them causes a real bug (not just a perf issue).

Answer: Beyond performance, these matter for **referential stability** — if a function or object is passed as a dependency to another hook, or as a prop to a `React.memo`-wrapped child, a new reference every render can cause infinite effect loops or unnecessary re-renders.

```
jsx
```

```
function Parent() {
  const [count, setCount] = useState(0);

  // Without useCallback, this is a NEW function every render,
  // which would re-trigger the effect in Child every time Parent re-renders
  const handleClick = useCallback(() => console.log('clicked'), []);

  return <Child onAction={handleClick} />;
}

const Child = React.memo(({ onAction }) => {
  useEffect(() => {
    // runs every time `onAction` reference changes
  }, [onAction]);
  return <button onClick={onAction}>Go</button>;
});
```

Q12: What is prop drilling, and what are the alternatives?

Answer: Prop drilling is passing a prop through several layers of components that don't need it themselves, just to reach a deeply nested child that does. Alternatives: **Context** for low-frequency shared data (theme, auth user); **component composition** (passing components as `children`/props instead of drilling primitive values down); state management libraries (Redux/Zustand/Jotai) for complex, frequently-updated shared state.

Q13: What is React reconciliation, and how does the diffing algorithm decide what to re-render?

Answer: React compares the new virtual DOM tree to the previous one using heuristics (not a full generic tree-diff, which would be too slow at $O(n^3)$ — React's algorithm is $O(n)$): elements of different types are assumed completely different (old subtree torn down, new one built); elements of the same type are compared prop-by-prop and only differences are applied; lists are matched by `key`. Understanding this explains *why* keys and component type stability matter so much for performance.

Q14: What's the difference between `React.memo`, `useMemo`, and `useCallback`?

Answer: `React.memo` wraps a *component* and skips re-rendering it if its props are shallow-equal to the last render. `useMemo` memoizes a *computed value* inside a component (skip recalculation if dependencies haven't changed). `useCallback` memoizes a *function reference* itself (functionally `useMemo(() => fn, deps)` under the hood) — mainly useful to keep that reference stable for `React.memo` children or effect dependencies.

Q15: Live-coding staple: build a debounced search filter component.

```
jsx

function useDebounceValue(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debounced;
}

function SearchableList({ items }) {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounceValue(query, 300);

  const filtered = useMemo(
    () => items.filter(i => i.name.toLowerCase().includes(debouncedQuery.toLowerCase())),
    [items, debouncedQuery]
  );

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} placeholder="Search" />
      <ul>{filtered.map(i => <li key={i.id}>{i.name}</li>)}</ul>
    </div>
  );
}
```

Talk through out loud while coding: why debounce lives in a hook (reusable), why

`useMemo` wraps the filter (avoid recompute on every unrelated re-render), why the `<input>` itself stays uncontrolled-by-debounce (typing feels instant; only filtering is delayed).

Rapid-Fire Round

- **Q: What's the Rules of Hooks?** A: Only call hooks at the top level (never in loops/conditions/nested functions) and only from React function components or custom hooks — this lets React track hook order/state correctly between renders.
- **Q: What's a custom hook?** A: A function starting with `use` that composes other hooks to encapsulate reusable stateful logic (`useFetch`, `useDebounceValue`, `useLocalStorage`).

- **Q: What is Strict Mode in React, and why does my `useEffect` run twice in dev?** A: `<React.StrictMode>` intentionally double-invokes certain functions (including effects) in development only, to help surface effects that aren't properly cleaned up — it does not happen in production.
 - **Q: What's the difference between state and props?** A: Props are passed in from a parent and are read-only from the child's perspective; state is owned and managed internally by the component itself.
 - **Q: What is lifting state up?** A: Moving shared state to the closest common ancestor of the components that need it, so they stay in sync via shared props rather than each having disconnected local state.
 - **Q: What's a fragment (`<>...</>`) for?** A: Lets you group multiple elements without adding an extra wrapper `<div>` to the DOM.
-

End of Part 2. Say "next part" for Part 3: Advanced React & Next.js (App Router, Pages Router, RSC, Hydration).